

PHASE 5

Vom lokalen Ticket-System in die Google Cloud

Ein ausführlicher Explainer zu Terraform, GCP, GKE und dem Weg zu einem belastbaren Cloud-Deployment

Repo: [high-frequency-ticket-system](#)

Stand: main @ f45f45b, 29. Juli 2026

Perspektive: Architektur + Lernpfad + konkrete Restarbeit

Die Leitidee

Phase 5 ist kein Umschreiben der Anwendung. Wir ersetzen lokale Infrastruktur-Simulation durch echte, gemanagte Cloud-Dienste und geben Kubernetes einen präzisen Betriebsvertrag: Images, Konfiguration, Identität, Netzwerk, Health, Skalierung und Rollout.

So lernst du mit diesem Dokument

Der Explainer folgt vier wirksamen Lernstrategien: mentale Modelle vor Syntax, Mapping vom bereits Bekannten zum Neuen, worked examples mit konkreten Repo-Bezügen und aktive Abrufübungen nach jedem großen Abschnitt. Lies zuerst Kapitel 1 bis 4. Danach kannst du Kapitel 5 bis 9 als Implementierungsleitfaden verwenden.

Strategie	Wie sie hier eingesetzt wird	Warum sie hilft
Chunking	Phase 5 wird in kleine Gates zerlegt.	Cloud-Komplexität bleibt lokal verständlich.
Elaboration	Jedes Produkt wird über seine Verantwortung erklärt.	Du lernst Entscheidungen statt Produktnamen.
Dual Coding	Architektur- und Abhängigkeitsdiagramme ergänzen Prosa.	Beziehungen werden sichtbar.
Retrieval	Checkfragen ohne direkte Lösung zwingen zum Erinnern.	Wissen wird abrufbar, nicht nur vertraut.
Fading	Erst vollständige Beispiele, später selbstständige Aufgaben.	Die Stütze wird schrittweise entfernt.

Inhalt

So lernst du mit diesem Dokument	2
Inhalt	3
1. Der aktuelle Stand des Repos	5
1.1 Was bereits cloudnah ist	5
1.2 Was Phase 5 offiziell fordert	5
1.3 Verdeckte Cloud-Blocker aus dem Code	6
2. Das mentale Modell: Was wandert eigentlich in die Cloud?	7
2.1 Das lokale Mapping	7
2.2 Control Plane und Data Plane	8
2.3 Was bleibt bewusst außerhalb von Kubernetes?	8
3. Terraform verstehen, bevor wir Terraform schreiben	9
3.1 Der Kernzyklus	9
3.2 Bootstrap-Paradox	9
3.3 Empfohlene Struktur für dieses Repo	10
3.4 Terraform-Ressourcen in Abhängigkeitsreihenfolge	10
3.5 Terraform-Sicherheitsregeln	10
4. Die GCP-Produkte und ihre Rolle	11
4.1 VPC: das private Rückgrat	11
4.2 GKE: der Workload-Orchestrator	11
4.3 Cloud SQL for PostgreSQL	11
4.4 Memorystore for Redis	12
4.5 Pub/Sub	12
4.6 Artifact Registry und Supply Chain	12
4.7 Identität und Secrets	12
4.8 Edge, DNS und TLS	12
4.9 Observability	13
5. Kubernetes so gut wie möglich nutzen	14
5.1 Das Objektmodell pro Service	14
5.2 Probes mit echter Semantik	14
5.3 Ressourcen und Autoscaling	14
5.4 Reconcile-HA: konkrete Empfehlung	15
5.5 Sale-Unlock: eine Uhr für alle Pods	15
5.6 Network Policies	15
6. Vom Monorepo zu drei sicheren Images	17
6.1 Multi-stage Build	17

6.2 Image-Checkliste	17
6.3 Das Next.js-Sonderproblem	17
6.4 Migration Image/Job	18
7. Der konkrete Implementierungsplan für Phase 5	19
5.0 - Cloud-Readiness im Code	19
5.1 - Containerisierung	19
5.2 - Terraform Bootstrap	19
5.3 - Netzwerk und Basisdienste	19
5.4 - Stateful Managed Services	19
5.5 - GKE und Kubernetes Baseline	20
5.6 - Reconcile-HA und Autoscaling	20
5.7 - CI/CD	20
5.8 - Cloud-Lasttest und Evidence	20
8. CI/CD, Betrieb und der echte Kapazitätsnachweis	21
8.1 Deployment-Pipeline	21
8.2 Rollback ist mehr als `kubect! rollout undo`	21
8.3 Cloud-Lasttest-Topologie	21
8.4 Was ein gültiger Run beweisen muss	21
8.5 Skalierungsdenken für den Sale	22
8.6 Kostenkontrolle	22
9. Priorisierte Restarbeit und Definition of Done	23
9.1 Kritischer Pfad	23
9.2 Was vor Phase 5 noch aus Phase 4 offen ist	23
9.3 Definition of Done für Phase 5	23
9.4 Meine Architekturentscheidung in einem Satz	23
10. Lernlabor: Aufgaben mit abnehmender Hilfestellung	24
Level 1 - Erklären	24
Level 2 - Anwenden	24
Level 3 - Entscheiden	24
Level 4 - Beweisen	24
Selbsttest	25
11. Glossar	26
12. Quellen und Repo-Belege	27
12.1 Repo-Belege	27
12.2 Offizielle Dokumentation	27

1. Der aktuelle Stand des Repos

Die Anwendung ist fachlich und messtechnisch weit fortgeschritten, aber cloudseitig noch am Startpunkt. Phase 4.7 (Reserve -> Pay -> Publish) ist abgeschlossen. Der DB-Hot-Row wurde entfernt: der Micro-Benchmark stieg laut Roadmap von 235 auf 26.385 Tickets/s. Baseline C persistierte 957.053 Orders fehlerfrei; die Messung war wegen 21,85 % dropped iterations jedoch kein gültiger Kapazitätsnachweis. Der lokale Connection-Pool und der co-lokalisierte Lastgenerator waren die sichtbaren Grenzen. [R2][R6]

Kurzturteil

Application readiness: hoch. Async Flow, Idempotenz, Redis-Lua, Pub/Sub-Consumer, Reconcile und Prometheus-Metriken existieren.

Platform readiness: niedrig. Terraform ist leer, Images fehlen, Kubernetes-Manifeste fehlen, Deployment-CI fehlt und mehrere lokale Annahmen brechen in Produktion.

1.1 Was bereits cloudnah ist

Baustein	Heute im Repo	Warum das für GCP gut ist
API	Fastify, stateless, Redis Reads, Pub/Sub Publish	Lässt sich horizontal skalieren, ohne DB im Hot Path.
Worker	Pull-Subscriber, Flow Control, idempotente DB-Transaktion	Passt direkt zu Pub/Sub At-Least-Once Delivery.
Datenbank	PostgreSQL + Drizzle + versionierte Migrationen	Cloud SQL kann denselben SQL-Vertrag hosten.
Cache	ioredis + atomare Lua-Skripte	Memorystore ist Redis-protokollkompatibel.
Observability	Prometheus-Endpunkte und Grafana-Dashboards	Managed Prometheus kann dieselben Metriken scrapen.
Konfiguration	Zod-validierte Env-Variablen	Fehlende Konfiguration stoppt Pods früh und sichtbar.
Logging	Pino JSON mit GCP severity	stdout landet strukturiert in Cloud Logging.

1.2 Was Phase 5 offiziell fordert

- Terraform für VPC, Cloud SQL, Memorystore und GKE.
- Dockerfiles für API, Worker und Web.
- Kubernetes Deployment-, Service- und Ingress-Manifeste.
- Cloud-Lasttest plus Metriken für die README.
- Eine podübergreifend verlässliche Sale-Unlock-Zeitquelle.
- Eine HA-Strategie für den Reconcile-Loop bei mehr als einem Worker.

1.3 Verdeckte Cloud-Blocker aus dem Code

Priorität	Befund	Warum gefährlich	Nötige Änderung
P0	'PUBSUB_EMULATOR_HOST' ist Pflicht.	In Produktion darf die Client-Library keinen Emulator-Host sehen.	Nur in development/test erlauben; in production muss die Variable fehlen.
P0	Öffentliches 'POST /reset' setzt Redis auf 1 Mio.	Jeder Internet-Client könnte den Bestand manipulieren.	Route in production deaktivieren oder strikt admin-authentisieren.
P0	CORS ist 'origin: true'.	Beliebige Origins dürfen Browser-Requests senden.	Explizite Origin-Allowlist aus Env.
P0	Nur '/health', ohne Dependency-Check.	Kubernetes kann einen lebenden, aber unbrauchbaren Pod routen.	'/livez', '/readyz', Startup-Probe und klare Semantik.
P0	Kein Cloud-Migrations-/Seed-Pfad.	Ein leerer Cloud-SQL-Server kann die App nicht bedienen.	Migration Job und getrennte Event-Initialisierung.
P1	Web-Konfiguration ist 'NEXT_PUBLIC_*'.	Werte werden typischerweise beim Next-Build eingebrannt.	Build pro Environment oder Runtime-Config-Pattern.
P1	Redis-URL wird geloggt.	AUTH-Credentials könnten in Cloud Logging landen.	URL redigieren; nie Secret-Werte loggen.
P1	Redis-Fehlertext nennt Docker Compose.	Produktionsdiagnose führt in die falsche Richtung.	Umgebungsneutrale Fehlermeldung.
P1	Worker reconciled in jedem Prozess.	Mehrere Worker schreiben konkurrierend denselben Desired State.	Kubernetes Lease Leader Election.
P1	Sale-Gate nutzt 'Date.now()' aus dem API-Pod.	Pod-Uhrdrift kann Unlock je Replica verschieben.	Redis 'TIME' im Lua-Skript als autoritative Uhr.

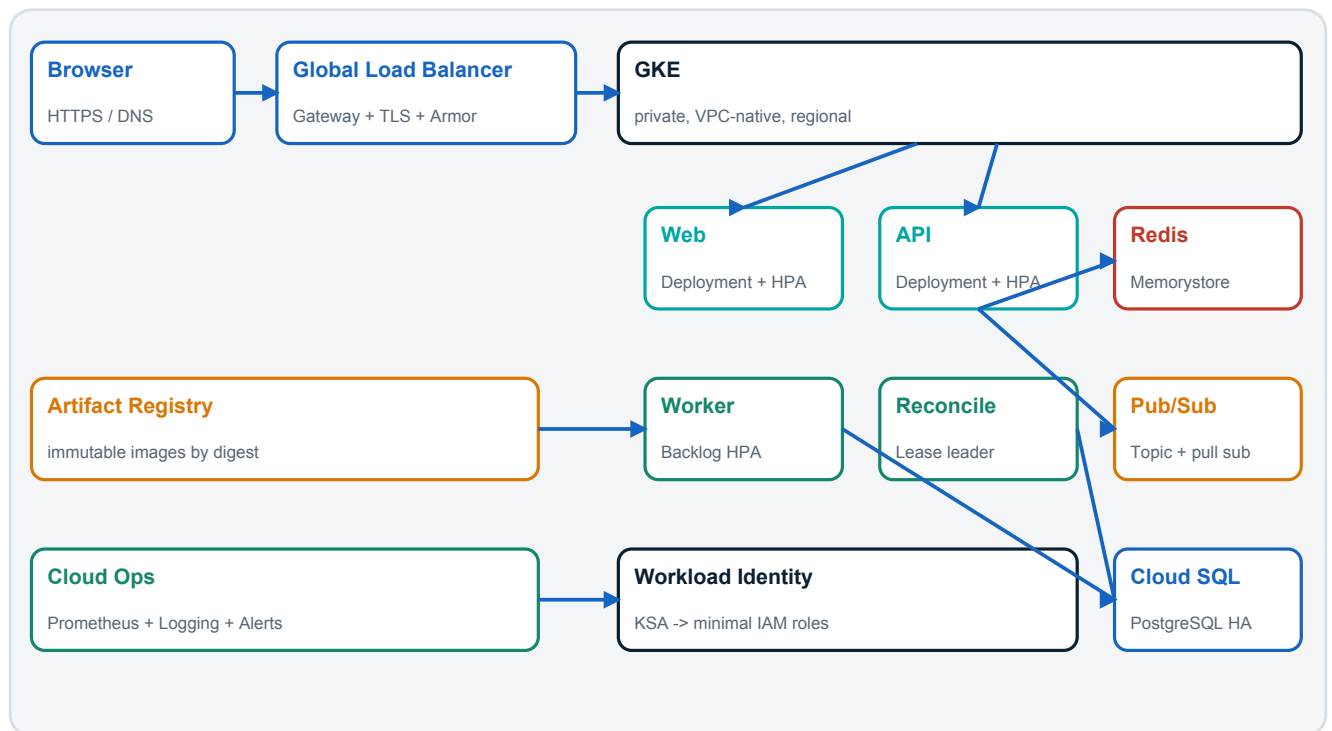
Abrufübung 1

Ohne zurückzublättern: Welche drei Eigenschaften machen die API horizontal skalierbar? Welche zwei Zustände verhindern aktuell gefahrloses Worker-Scaling?

2. Das mentale Modell: Was wandert eigentlich in die Cloud?

Lokal übernimmt Docker Compose mehrere Rollen gleichzeitig: Netzwerk, Service Discovery, Prozessstart, persistente Volumes und Test-Infrastruktur. In GCP werden diese Verantwortungen bewusst getrennt. Die Cloud ist nicht 'Docker Compose auf einem großen Server', sondern eine Sammlung von Kontrollsystemen.

Denke in drei Ebenen: Terraform erzeugt die langlebige Plattform, Kubernetes betreibt die kurzlebigen App-Workloads, und gemanagte GCP-Dienste halten den kritischen Zustand. Diese Trennung ist der Kern der Phase.



2.1 Das lokale Mapping

Lokal	Cloud-Ziel	Verantwortung
Docker-Netzwerk	Custom VPC + Subnet + Pod/Service Ranges	Private Erreichbarkeit und IP-Plan.
postgres:16	Cloud SQL for PostgreSQL	Persistenz, HA, Backup, PITR.
redis:7	Memorystore for Redis	Hot Read Model, Ledger, atomare Reservierung.
Pub/Sub Emulator	Google Cloud Pub/Sub	Puffer, Backpressure, Redelivery.
Native API/Worker-Prozesse	GKE Deployments	Replikation, Rollout, Self-Healing.
Next dev/start	GKE Web Deployment	Frontend-Auslieferung.
Prometheus Container	Managed Service for Prometheus	Scraping und langfristige Metrics.
Grafana Container	Grafana mit Cloud Monitoring Datasource oder Cloud Dashboards	Visualisierung.
`.env`	ConfigMap + Secret Manager + Workload Identity	Konfiguration und geheime Werte.
`pnpm seed`	Migration Job + kontrollierter Admin/Loadtest-Seed	Schema und Testdaten getrennt.

2.2 Control Plane und Data Plane

Control Plane: Terraform, GKE Controller und Kubernetes API beschreiben und korrigieren Sollzustand. Data Plane: Web-, API- und Worker-Pods verarbeiten echte Requests und Nachrichten. Ein guter Betrieb mischt diese Ebenen nicht: Ein App-Pod provisioniert kein Topic, und Terraform verarbeitet keine Order.

Transfer Learning

Der vorhandene Reconcile-Loop ist bereits ein Kubernetes-Controller im Kleinen: Er vergleicht Sollzustand (Kapazität minus Tickets minus Reservierungen) mit Istzustand (Redis available) und korrigiert Drift. Kubernetes macht dasselbe mit Deployments und Pods.

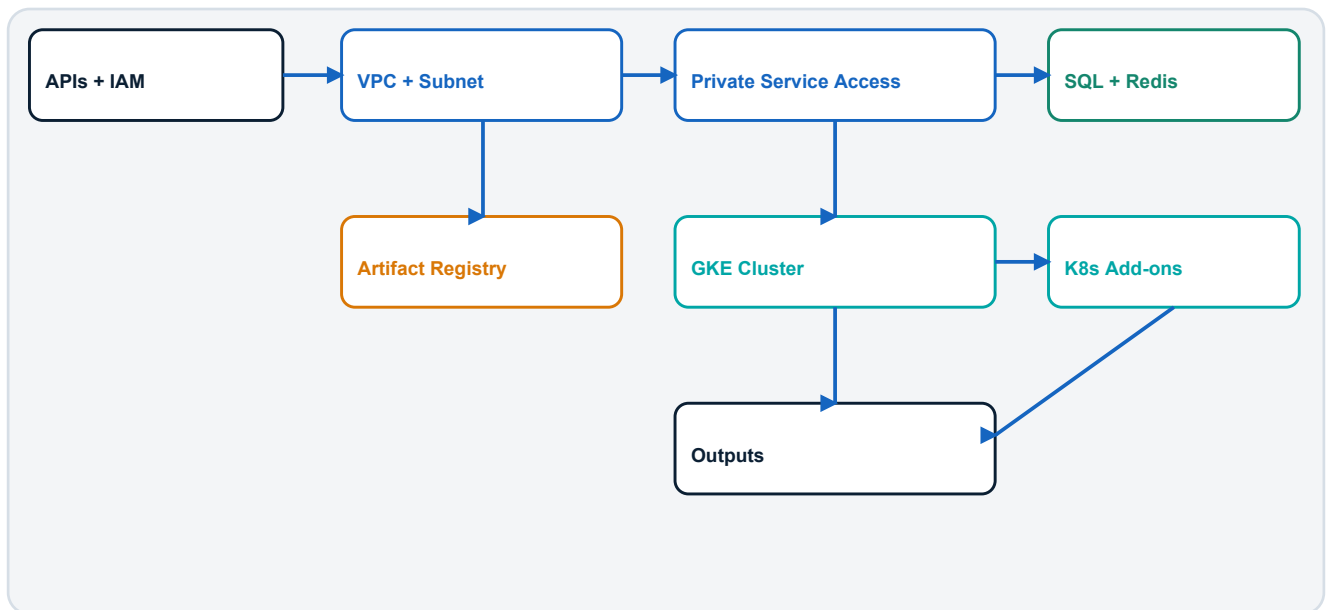
2.3 Was bleibt bewusst außerhalb von Kubernetes?

- PostgreSQL bleibt in Cloud SQL. Ein StatefulSet würde Backups, Failover, Patching und Storage-Betrieb wieder zu unserer Aufgabe machen.
- Redis bleibt in Memorystore. Der Ticket-Hot-Path braucht Redis, aber nicht die Lernaufgabe, Redis-HA selbst zu betreiben.
- Pub/Sub bleibt ein global gemanagter Broker. Kubernetes soll Consumer skalieren, nicht den Broker nachbauen.
- Container-Images liegen in Artifact Registry. Pods referenzieren unveränderliche Digests.

3. Terraform verstehen, bevor wir Terraform schreiben

Terraform ist ein deklarativer Planer für langlebige Infrastruktur. Du beschreibst Ressourcen und Beziehungen; Terraform liest den existierenden Zustand, berechnet einen Plan und ruft Provider-APIs auf. Es ist kein Shell-Skript und kein Kubernetes-Deployment-Tool.

Die wichtigste Datei ist nicht `main.tf`, sondern der State. Er verbindet eine Terraform-Adresse wie `google\_sql\_database\_instance.main` mit einer realen GCP-Ressource. Deshalb gehört State niemals ins Git-Repo. Google empfiehlt für Terraform auf GCP einen Cloud-Storage-Backend. [GCP-8]



3.1 Der Kernzyklus

```

terraform fmt -check
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan

```

`plan` ist eine Change-Review. `apply` führt exakt den gespeicherten Plan aus. In CI sollte ein Pull Request formatieren, validieren und planen; ein geschützter Deployment-Job darf nach Review anwenden.

3.2 Bootstrap-Paradox

Der GCS-Bucket, der Terraform-State hält, kann nicht zuverlässig vom selben State verwaltet werden. Deshalb brauchen wir einen kleinen einmaligen Bootstrap-Stack oder ein dokumentiertes `gcloud`-Bootstrap. Danach verwaltet der Haupt-Stack alle normalen Ressourcen.

3.3 Empfohlene Struktur für dieses Repo

```

terraform/
  bootstrap/                # State-Bucket + Basis-IAM, einmalig
  modules/
    network/
      gke/
      cloud-sql/
      memorystore/
      pubsub/
      artifact-registry/
      observability/
  environments/
    dev/
      main.tf
      variables.tf
      outputs.tf
      backend.hcl.example
      terraform.tfvars.example
      loadtest/
      ...
  k8s/
    base/
    overlays/dev/
    overlays/loadtest/

```

Für ein einzelnes Showcase-Projekt ist das genug Modularität. Ein Modul pro einzelner Resource wäre Abstraktion ohne Gewinn. Die Environment-Verzeichnisse besitzen getrennte States, sodass ein `dev`-Destroy nicht versehentlich die Loadtest-Umgebung trifft.

3.4 Terraform-Ressourcen in Abhängigkeitsreihenfolge

Welle	Ressourcen	Warum diese Reihenfolge
0	Project APIs, Service Accounts, State	Ohne APIs und Identitäten kann nichts anderes entstehen.
1	VPC, Subnet, sekundäre Ranges, PSA, NAT	SQL, Redis und GKE hängen am Netzwerk.
2	Artifact Registry, Pub/Sub, Secret Container	Unabhängige Plattformdienste.
3	Cloud SQL, Memorystore	Private IPs benötigen PSA und VPC.
4	GKE Cluster + Workload Identity	Cluster hängt am Subnet und IAM.
5	Kubernetes Baseline-Objekte	Erst nach Cluster und Image Registry.
6	Gateway, DNS, TLS, Armor, Alerts	Braucht Services und öffentliche Endpunkte.

3.5 Terraform-Sicherheitsregeln

- Keine Service-Account-JSON-Keys. GitHub Actions und GKE nutzen Workload Identity Federation. [GCP-7][GCP-9]
- Keine Secret-Payload in `tfvars`: Terraform-State kann Werte enthalten, auch wenn Ausgaben als sensitive markiert sind.
- Least Privilege: API darf Pub/Sub publizieren; Worker darf abonnieren und Cloud SQL verbinden; nicht beide alles.
- Plan und Apply mit getrennten Berechtigungen; Production Apply nur auf geschützter Branch/Environment.
- Deletion Protection für Cloud SQL und optional GKE; Budget Alerts und Labels auf jede Resource.

Abrufübung 2

Warum ist Terraform-State sicherheitsrelevant? Warum braucht Private Service Access eine eigene frühe Welle? Was gehört absichtlich nicht in Terraform-State?

4. Die GCP-Produkte und ihre Rolle

4.1 VPC: das private Rückgrat

Wir verwenden ein Custom-Mode-VPC, kein Default-Netz. Ein regionales Subnet erhält einen primären Range für Nodes sowie sekundäre Ranges für Pods und Services. GKE wird VPC-native. Private Google Access erlaubt privaten Nodes den Zugriff auf Google APIs; Cloud NAT deckt kontrollierten Egress ab.

Cloud SQL und klassisches Memorystore for Redis nutzen private Erreichbarkeit. Dafür reservieren wir einen separaten Private-Service-Access-Range und verbinden ihn über Service Networking. GKE und Redis müssen im autorisierten Netzwerk liegen; für Redis mit Private Service Access ist VPC-native/IP-Aliasing erforderlich. [GCP-3][GCP-4][GCP-5]

IP-Plan zuerst

Ein schlechter CIDR-Plan ist später teuer. Plane getrennte, nicht überlappende Ranges für Nodes, Pods, Services und Managed Services. Dokumentiere Region und CIDRs als Architekturvertrag, nicht als zufällige Terraform-Defaults.

4.2 GKE: der Workload-Orchestrator

GKE betreibt unsere drei App-Komponenten. Autopilot verwaltet Nodes, Skalierung und viele Security-Defaults und ist für die meisten Production-Workloads optimiert. [GCP-1] Für dieses Projekt empfehle ich dennoch GKE Standard, regional für den belastbaren 50k-RPS-Lernpfad: Wir wollen Kapazität vor einem bekannten Sale vorwärmen, Node Pools verstehen und Lastgenerator und SUT bewusst trennen. Eine kleine Dev-Umgebung kann Autopilot nutzen, wenn Kosten und Einfachheit wichtiger sind.

Kriterium	Autopilot	Standard	Empfehlung hier
Node-Betrieb	Google entscheidet	Wir konfigurieren Pools	Standard für gezieltes Lernen.
Flash-Sale	Pods triggern Compute-Provisioning	Kapazität kann vorgewärmt werden	Standard + Mindestnodes vor Unlock.
Kosten	Pod-basiert, komfortabel	Idle Nodes kosten	Dev klein, Loadtest zeitlich begrenzt.
Security Defaults	stark vorkonfiguriert	mehr Eigenverantwortung	Private Nodes + Shielded + WI.
K8s-Lernfläche	Workload-fokussiert	Workload + Nodes	Standard zeigt die ganze Skalierungskette.

4.3 Cloud SQL for PostgreSQL

Cloud SQL ersetzt den lokalen PostgreSQL-Container. Production-ähnlich bedeutet: private IP, regionale HA, automatische Backups, Point-in-Time Recovery, Maintenance Window und Deletion Protection. Regionale HA schützt gegen den Ausfall einer Zone; Google empfiehlt automatisierte Backups und PITR. [GCP-10][GCP-11]

Der Worker verbindet über den Cloud SQL Auth Proxy als Sidecar auf `127.0.0.1:5432`. Der Proxy nutzt Workload Identity, TLS und IAM-Autorisierung; GCP empfiehlt das Sidecar-Muster für GKE. [GCP-2] Die API bekommt bewusst keine Datenbankberechtigung - damit wird die Architekturregel technisch erzwungen.

Connection-Budget statt blindem HPA

Der lokale Befund zeigt: Pool-Größe ist ein Durchsatzhebel. In Kubernetes gilt aber `max DB connections = Worker replicas x DATABASE\_POOL\_MAX + Jobs + Reserve`. Ein HPA, der Worker hochskaliert, kann Cloud SQL durch zu viele Connections überlasten. Lege zuerst ein globales Connection-Budget fest; skaliere dann Pods und Pool gemeinsam. PgBouncer ist erst nötig, wenn dieses Budget zum echten Engpass wird.

4.4 Memorystore for Redis

Memorystore ist die exklusive Read-Quelle der API und hält Inventory, Reservation Ledger und Order-Cache. Für production-like Betrieb: Standard Tier für HA, private IP, AUTH und eine bewusste Memory-/Eviction-Policy. Memorystore ist Redis-protokollkompatibel, sodass die vorhandenen ioredis-Operationen weiter funktionieren. [GCP-6]

Korrektheitsgrenze

Order-Cache darf auslaufen; das Reservation Ledger darf nicht willkürlich evicted werden. Prüfe die Memorystore-Maxmemory-Policy gegen diesen Datenvertrag. Phase 6 muss zudem verwaiste Reservierungen reapen, sonst bindet ein Abbruch Inventar unbegrenzt.

4.5 Pub/Sub

Terraform provisioniert Topic und Pull-Subscription einmalig. Die Anwendung bleibt Runtime-Client. Wir behalten At-Least-Once Delivery: Die Datenbanktransaktion ist über `orderId` idempotent und der processed-Marker ist nur eine Optimierung. Exactly-once kann die Latenz erhöhen und ersetzt keine Business-Idempotenz; es ist regional und nur für Pull/StreamingPull verfügbar. [GCP-12]

Phase 5 braucht Topic, Subscription, Ack-Deadline passend zur maximalen Verarbeitung, Retention und IAM. Retry Policy und Dead Letter Topic gehören in Phase 6, sollten aber im Terraform-Modul bereits erweiterbar sein.

4.6 Artifact Registry und Supply Chain

API, Worker und Web werden als OCI-Images in einer regionalen Artifact Registry gespeichert. Artifact Registry ist der empfohlene GCP-Registry-Dienst. Images werden mit Commit SHA getaggt und im Deployment per Digest referenziert; Digests sind unveränderlich. [GCP-13][GCP-14]

4.7 Identität und Secrets

Workload Identity Federation for GKE gibt Kubernetes Service Accounts gezielte GCP-Rechte, ohne JSON-Key-Dateien. In Autopilot ist sie immer aktiv; in Standard wird sie am Cluster und an Node Pools aktiviert. [GCP-7] Wir brauchen mindestens `api`, `worker`, `migrator` und `reconciler` als getrennte Kubernetes-Identitäten.

KSA	GCP-Rechte	Explizit nicht
api	Pub/Sub Publisher; Secret Access nur falls nötig	Cloud SQL Client, Subscriber
worker	Pub/Sub Subscriber; Cloud SQL Client; Secret Access	Publisher, Infra Admin
migrator	Cloud SQL Client; DB-Migrationssecret	Pub/Sub, Redis Admin
reconciler/leader	Cloud SQL Client; Redis Runtime-Zugang; K8s Lease RBAC	Cloud-Projekt-Admin
github-deployer	Artifact Writer + eingeschränktes GKE Deploy	Service-Account-Key, Owner

Secret Manager speichert DB-Credentials. Das GKE Secret Manager Add-on kann Secrets als Volumes mounten und wird von Google verwaltet. [GCP-15] Da die App heute eine vollständige `DATABASE\_URL` als Env erwartet, müssen wir entweder Secret-Sync zu einem Kubernetes Secret verwenden oder den Config-Vertrag auf einzelne Werte plus `\*\_FILE` erweitern. Die zweite Variante ist klarer und rotiert besser.

4.8 Edge, DNS und TLS

Cloud DNS zeigt die Domain auf eine reservierte globale IP. GKE Gateway API erzeugt den externen Application Load Balancer. HTTPRoutes schicken `api/\*` zur API und `/` zum Web. Certificate Manager liefert ein Google-managed Zertifikat. Gateway API ist der modernere, rollenorientierte Nachfolger von Ingress; Google empfiehlt für globale externe Load Balancer die `gke-l7-global-external-managed` Klasse. [GCP-16]

Cloud Armor sitzt am Load Balancer: zunächst Managed WAF-Regeln und grobes Rate-Limiting. Fachliches Rate-Limiting und Bot-/Queue-Management bleiben Phase 6. Ein wichtiges Detail: Der öffentliche Reset-Endpunkt darf niemals über die HTTPRoute erreichbar sein.

4.9 Observability

Cloud Logging sammelt stdout/stderr der Pods. Managed Service for Prometheus scraped die vorhandenen `/metrics`-Endpunkte über `PodMonitoring`-Ressourcen und speichert Prometheus-kompatible Daten im globalen Cloud-Monitoring-Backend. Google empfiehlt managed collection; vorhandene PromQL- und Grafana-Workflows bleiben nutzbar. [GCP-17][GCP-18]

Für die API skaliert HPA zunächst mit CPU plus Load-Balancer-Traffic; GKE unterstützt Gateway-Traffic als führendes Signal. Für den Worker ist Pub/Sub Backlog das bessere externe Signal als CPU. HPA kann mehrere Metriken kombinieren und nimmt die größte berechnete Replica-Zahl. [GCP-19][GCP-20]

5. Kubernetes so gut wie möglich nutzen

Kubernetes ist dann gut genutzt, wenn jedes Objekt eine konkrete Betriebsfrage beantwortet. Viele YAML-Dateien sind kein Qualitätsmerkmal. Für dieses System brauchen wir einen kleinen, vollständigen Satz.

5.1 Das Objektmodell pro Service

Objekt	Frage, die es beantwortet	Für dieses Repo
Deployment	Wie viele Pods, welches Image, wie rollen Updates?	web, api, worker
Service	Wie finden andere Komponenten stabile Endpoints?	web + api; Worker meist ohne normalen Service
Gateway + HTTPRoute	Wie kommt HTTPS-Traffic hinein?	Path Routing zu web/api
ConfigMap	Welche nicht geheimen Werte gelten?	Topic, Subscription, Log Level, TTLs
Secret/CSI	Wie kommen geheime Werte hinein?	DB-Credentials, ggf. Redis AUTH
Kubernetes ServiceAccount	Welche Identität hat der Pod?	separat pro Rolle
HPA	Wann entstehen mehr Pods?	API/Web; Worker erst nach Reconcile-HA
PDB	Wie viele Pods dürfen freiwillig gleichzeitig fehlen?	API/Web; Worker nach HA
PodMonitoring	Welche Metrics werden gescraped?	API + Worker `/metrics`
NetworkPolicy	Wer darf wohin sprechen?	Default deny + explizite Egress/Ingress
Lease	Wer ist der aktive Reconcile-Leader?	ein Leader unter mehreren Workern
Job	Wie laufen endliche Admin-Aufgaben?	DB-Migration, kontrollierter Seed

5.2 Probes mit echter Semantik

Probe	Bedeutung	Empfehlung
startupProbe	Hat der Prozess seine Initialisierung geschafft?	Großzügig; schützt langsamen Start vor Liveness-Kills.
livenessProbe	Ist der Event Loop grundsätzlich lebendig?	`/livez`, keine externen Abhängigkeiten.
readinessProbe	Kann dieser Pod jetzt Traffic/Arbeit übernehmen?	API: Redis; Worker: Redis + DB + Subscriber registriert.

Ein Readiness-Fehler nimmt einen API-Pod aus dem Load Balancer, startet ihn aber nicht sofort neu. Ein Liveness-Fehler startet neu. Wenn Liveness direkt Redis prüft, kann ein Redis-Ausfall alle Pods in eine Restart-Spirale schicken - daher die Trennung.

5.3 Ressourcen und Autoscaling

Jeder Container braucht CPU-/Memory-Requests. HPA-Prozentwerte funktionieren nur sinnvoll, wenn Requests gesetzt sind. [GCP-19] Limits werden vorsichtig gewählt: zu enge CPU-Limits erzeugen Throttling im Peak; Memory-Limits schützen Nodes vor einem unbounded Prozess.

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    memory: 512Mi
```

Die Zahlen sind Startwerte, keine Wahrheit. Erst ein Cloud-Baseline-Lauf liefert VPA-Empfehlungen und HPA-Ziele. Vor dem angekündigten Sale wird die Mindestzahl der API-Pods und Standard-Nodes hochgesetzt; Autoscaling ist die

zweite Verteidigungslinie, nicht die einzige.

API

- MinReplicas ≥ 3 über mehrere Zonen; HPA auf CPU und Gateway-Traffic/RPS.
- RollingUpdate mit `maxUnavailable: 0`, `maxSurge` > 0 ; PDB und Topology Spread.
- PreStop + ausreichende terminationGracePeriod, damit Keep-Alive Requests sauber enden.

Worker

- Skalierung nach unacked Pub/Sub messages und oldest unacked age, mit DB-Connection-Budget als Obergrenze.
- Flow Control pro Pod bleibt aktiv. HPA x maxMessages x DB pool muss zusammen dimensioniert werden.
- Reconcile läuft nur im Lease-Leader. Alle Worker dürfen weiterhin Nachrichten verarbeiten.

Web

- Stateless Deployment, kleine Ressourcen, HPA auf CPU oder Gateway-Traffic.
- Same-origin Routing bevorzugen, damit CORS fast vollständig entfällt.

5.4 Reconcile-HA: konkrete Empfehlung

Ich empfehle Option A: Kubernetes Lease Leader Election. Jeder Worker-Pod versucht, ein `coordination.k8s.io/v1 Lease` zu halten. Nur der aktuelle Leader startet Startup-Reconcile und Timer. Verliert er die Lease, stoppt er den Timer. Fällt er aus, übernimmt ein anderer Pod nach Ablauf der Lease.

Entscheidung	Vorteil	Risiko/Gegenmaßnahme
Lease in Worker	Keine zusätzliche Deployable Unit; echte Failover-HA.	Mehr Code und RBAC; gut isolierbar und testbar.
Dedizierter Singleton	Klare Verantwortung.	Replica 1 ist nicht HA; braucht für echte HA wiederum Leader Election.
CronJob	Einfaches K8s-Objekt.	10-Sekunden-Peak-Intervall und überlappungsfreier Dauerbetrieb passen schlecht.

5.5 Sale-Unlock: eine Uhr für alle Pods

Der Lua-Hot-Path sollte die Zeit direkt aus Redis lesen. `redis.call("TIME")` liefert Sekunden und Mikrosekunden. Das Skript bildet daraus Millisekunden und verwendet denselben Wert als Ledger-Score. Damit entscheiden alle API-Pods auf derselben autoritativen Uhr.

```
local redisTime = redis.call("TIME")
local nowMs = tonumber(redisTime[1]) * 1000
+ math.floor(tonumber(redisTime[2]) / 1000)

local opensAt = tonumber(redis.call("GET", KEYS[4]) or "0")
if opensAt > 0 and nowMs < opensAt then
    return {-2, nowMs}
end
```

Der Rückgabewert kann `nowMs` enthalten, sodass `queuedAt` später konsistent gesetzt wird. Das erfordert eine kleine Änderung am TypeScript-Wrapper und neue Integrationstests für Vorher/Gleich/Nachher.

5.6 Network Policies

Beginne pro Namespace mit Default-Deny. Erlaube dann nur: Gateway -> Web/API, API -> Redis + Pub/Sub/Google APIs, Worker -> Redis + Pub/Sub + Cloud SQL Proxy, Managed Prometheus Collector -> Metrics-Ports. Beachte: Netzwerkpolicies allein modellieren GCP-API-Ziele oft nicht elegant; Private Google Access, DNS und Egress-Design müssen mitgedacht werden.

Abrufübung 3

Welche Probe darf Redis prüfen, welche nicht? Warum ist CPU allein ein schlechtes Worker-Skalierungssignal?
Warum löst ein dedizierter Reconciler mit Replica 1 das HA-Problem nicht vollständig?

6. Vom Monorepo zu drei sicheren Images

Ein Container-Image ist der unveränderliche Runtime-Vertrag zwischen Code und Kubernetes. Es muss ohne Workspace-Quellbaum starten, als Non-Root laufen, auf `0.0.0.0` lauschen und ein SIGTERM sauber behandeln.

6.1 Multi-stage Build

Für pnpm/Turborepo bauen wir in mehreren Stufen: Dependencies, Build und schlanke Runtime. Der Build-Kontext bleibt der Repo-Root, damit Workspace-Packages sichtbar sind. `pnpm deploy` oder ein gezielter Prune-Schritt erzeugt je App nur notwendige Runtime-Abhängigkeiten.

```
# Konzept, nicht Copy/Paste-final
FROM node:24-bookworm-slim AS build
WORKDIR /repo
RUN corepack enable
COPY pnpm-lock.yaml pnpm-workspace.yaml package.json ./
COPY apps ./apps
COPY packages ./packages
RUN pnpm install --frozen-lockfile
RUN pnpm --filter api... build

FROM node:24-bookworm-slim AS runtime
ENV NODE_ENV=production
USER node
WORKDIR /app
COPY --from=build --chown=node:node /repo/<pruned-runtime> ./
EXPOSE 10002
CMD ["fastify", "start", "-a", "0.0.0.0", "-p", "10002", "dist/app.js"]
```

6.2 Image-Checkliste

- Node-Version pinnen; Base Image per Digest fixieren und regelmäßig aktualisieren.
- `.dockerignore`: `.git`, `node\_modules`, `Coverage`, `Reports`, `Secrets`, `Terraform-State` ausschließen.
- Non-Root User, read-only root filesystem soweit möglich, `allowPrivilegeEscalation: false`.
- Kein `pnpm run build` beim Containerstart. Build geschieht einmal in CI.
- Keine DevDependencies und kein Source Map mit geheimen Buildpfaden, wenn nicht benötigt.
- OCI Labels: revision, source, created; Image-Scan in Artifact Analysis.
- Smoke-Test: Container starten, Health prüfen, SIGTERM senden, Exit-Code und Shutdown beobachten.

6.3 Das Next.js-Sonderproblem

`NEXT\_PUBLIC\_API\_URL` und `NEXT\_PUBLIC\_EVENT\_ID` werden im Browser verwendet und bei Next-Builds häufig eingebrannt. Das kollidiert mit dem Ideal 'ein Image für alle Environments'. Drei Optionen:

Option	Trade-off	Empfehlung
Image pro Environment	Einfach, aber mehrere Builds desselben Commits.	Für ersten Dev-Deploy akzeptabel.
Same-origin + relative API URL	Weniger CORS, stabiler Runtime-Vertrag.	Bevorzugt: `apiUrl` als `window.location.origin` oder leer.
Runtime config.js/Endpoint	Ein Image, dynamisch; etwas Zusatzcode.	Gut, wenn mehrere Environments wichtig werden.

Für dieses System ist Same-Origin am saubersten: Gateway routet `/api` zur API und den Rest zum Web. Damit braucht das Frontend keine separate öffentliche API-URL, und CORS kann auf die echte Domain beschränkt oder weitgehend entfallen.

6.4 Migration Image/Job

Schema-Migrationen dürfen nicht von jedem Worker beim Start ausgeführt werden. Baue einen endlichen Migrations-Entrypoint aus `@repo/db`; ein Kubernetes Job verbindet über Cloud SQL Auth Proxy, führt `drizzle-kit migrate` und die versionierten SQL-Migrationen aus und endet. Der App-Rollout startet erst nach erfolgreicher Migration. Für parallele Deployments braucht der Migrationspfad einen Advisory Lock oder eine CI-seitige Serialisierung.

7. Der konkrete Implementierungsplan für Phase 5

Die Reihenfolge minimiert die Zahl gleichzeitiger Unbekannter. Jede Stufe endet mit einem überprüfbaren Gate. Wir deployen nicht sofort alles auf einmal.

5.0 - Cloud-Readiness im Code

Task	Nachweis
Production-Env-Schema: Emulator-Host optional/verboten in production	Unit-Test für development/test/production.
`/reset` in production deaktivieren oder Admin-only	Production-Smoke liefert 404/403.
CORS-Allowlist oder Same-Origin	Fremde Origin wird abgelehnt.
`/livez`, `/readyz`, Startup-Semantik	Dependency-Ausfall beeinflusst nur Readiness.
Redis-URL-Logging redigieren	Log-Snapshot enthält keine Credentials.
Sale-Unlock auf Redis TIME	Integrationstests vor/auf/nach opensAt.
Graceful Shutdown und Server bind address	SIGTERM-Smoke im Container.

5.1 - Containerisierung

- Dockerfiles für web/api/worker plus `.dockerignore`.
- Lokaler Compose-Smoke mit den gebauten Images, nicht den Dev-Prozessen.
- SBOM/Vulnerability Scan und feste Image-Digests.
- Migrations-Entrypoint als eigenes Image-Ziel oder Command.

5.2 - Terraform Bootstrap

- GCP Project/Billing auswählen; Region festlegen, zum Beispiel `europe-west3` oder eine bewusst gewählte Alternative.
- GCS State Bucket mit Versioning, Uniform Bucket-Level Access und Public Access Prevention.
- GitHub OIDC Workload Identity Pool/Provider und eingeschränkte Plan-/Apply-Identitäten.
- Budget und Budget Alerts sofort anlegen - nicht erst nach dem ersten vergessenen Cluster.

5.3 - Netzwerk und Basisdienste

- Custom VPC, Subnet, Pod-/Service-Ranges, Private Google Access, Cloud NAT.
- Private Service Access Range + Service Networking Connection.
- Artifact Registry und Pub/Sub Topic/Subscription.
- Secret Manager Secret-Container ohne Secret-Payload im Terraform-Code.

5.4 - Stateful Managed Services

- Cloud SQL private IP, HA im loadtest/prod-like Environment, Backup + PITR, deletion protection.
- DB/User/Secret-Bootstrap; Cloud SQL Proxy-Verbindung aus einem Test-Pod.
- Memorystore Standard Tier, private Netzwerkverbindung, AUTH/TLS-Entscheidung, Memory Policy.
- Connectivity-Tests von einem temporären Pod; danach Debug-Pod wieder entfernen.

5.5 - GKE und Kubernetes Baseline

- Regionaler private Standard-Cluster, Workload Identity, Managed Prometheus, Gateway API.
- Namespace, KSAs, RBAC, ConfigMaps, SecretProviderClass/Secret Sync.
- Migration Job, danach Web/API/Worker Deployments und Services.
- Probes, Resources, SecurityContext, PDB, Topology Spread, Network Policies.
- Gateway, HTTPRoutes, DNS, TLS; `/reset` bleibt unerreichbar.

5.6 - Reconcile-HA und Autoscaling

- Lease Leader Election implementieren, Unit-/Integrationstests und RBAC.
- Worker auf mindestens 2 Replicas erhöhen; nur ein Leader reconciled.
- API/Web HPA kalibrieren; Worker HPA an Pub/Sub Backlog mit Connection-Budget.
- Sale-Prewarming Runbook: min replicas/nodes vor opensAt erhöhen.

5.7 - CI/CD

- CI: Quality Gates -> drei Images parallel bauen -> scannen -> nach Artifact Registry pushen.
- Images mit Commit SHA taggen und Digest erfassen; niemals `latest` deployen.
- Terraform Plan im PR; Apply und Kubernetes Deploy in geschütztem Environment.
- Migration Job -> canary/smoke -> Rollout -> Readiness/metrics prüfen -> automatische oder dokumentierte Rollback-Regel.

5.8 - Cloud-Lasttest und Evidence

- Loadgenerator außerhalb des SUT-Clusters oder in separatem Node Pool/Projekt.
- Mindestens circa 20k aktive VUs für das 50k-RPS-Ziel dimensionieren; 0 dropped iterations als Ziel.
- Cloudfähige Preflight-, Snapshot-, Seed- und Report-Adapter implementieren.
- Vorher/Nachher-Snapshots aus Cloud SQL, Memorystore, Pub/Sub und Managed Prometheus.
- Run-ID, Commit SHA, Image Digests, Terraform Outputs, GKE-Version und Kostenfenster archivieren.

8. CI/CD, Betrieb und der echte Kapazitätsnachweis

8.1 Deployment-Pipeline

```
Pull Request
-> lint / typecheck / tests / build
-> terraform fmt / validate / plan
-> Docker build + unit image smoke + scan

Merge main
-> push images by commit SHA
-> resolve immutable digests
-> terraform apply (protected)
-> migration Job
-> deploy manifests with digests
-> readiness + functional smoke
-> verify metrics and logs
```

GitHub Actions authentifiziert per OIDC/Workload Identity Federation, nicht per langem Service-Account-Key. Federation ist explizit für Deployment-Pipelines wie GitHub vorgesehen. [GCP-9]

8.2 Rollback ist mehr als `kubectl rollout undo`

Ein App-Rollback ist nur sicher, wenn die vorherige Version mit dem neuen DB-Schema kompatibel ist. Verwende Expand/Contract-Migrationen: zuerst additive Schemaänderung, dann Code-Rollout, erst später alte Spalten entfernen. Images per Digest machen das technische Rollback reproduzierbar.

8.3 Cloud-Lasttest-Topologie

Komponente	Platzierung	Grund
SUT	Privater GKE Cluster / dedizierte Node Pools	Saubere Anwendungs- und Cluster-Metriken.
k6 Runner	Separater Cluster, Managed Instance Group oder k6 Operator Node Pool	Generator konkurriert nicht um CPU/Netzwerk.
Metrics	Managed Prometheus + Cloud Monitoring	Skalierbare zentrale Zeitreihen.
Snapshots	Read-only Admin Job/Runner über private Connectivity	Kein `docker exec`; reproduzierbare Cloud-Adapter.
Artifacts	Cloud Storage oder versioniertes Report-Verzeichnis	Rohdaten bleiben prüfbar.

8.4 Was ein gültiger Run beweisen muss

- Generator: 0 dropped iterations oder eine vorab definierte enge Grenze; keine VU-/CPU-Sättigung.
- System: keine unerwarteten 5xx/Transportfehler; erwartete 425 und 409 separat.
- Korrektheit: published = completed + failed; DB orders/tickets konsistent; pending nach Drain = 0.
- Inventory: available + sold + aktive Reservierungen = Kapazität, inklusive signed drift.
- SLOs: API p95/p99, Publish->Persist p95/p99, oldest unacked age und Drain-Dauer.
- Ressourcen: Pod-/Node-Skalierung, DB Connections, Pool Wait, Redis Memory/Ops, Pub/Sub Backlog.
- Reproduzierbarkeit: Commit, Image Digests, Terraform-Version, Provider Lockfile, GKE-Version, Region und Konfiguration.

8.5 Skalierungsdenken für den Sale

Bei einem vorher bekannten Flash-Sale ist reaktives Autoscaling allein zu spät. Das Betriebsmodell hat drei Phasen: prewarm (Nodes, Pods, DB und Redis vorhalten), absorb (Pub/Sub puffert, API bleibt schnell), drain (Worker baut den Backlog innerhalb eines SLO ab). Das ist die eigentliche Stärke der Architektur.

Worked Example: Worker-Connection-Budget

Angenommen Cloud SQL erlaubt 400 App-Connections. 50 bleiben für Migration, Diagnose und Reserve. Bei `DATABASE_POOL_MAX=35` sind maximal 10 Worker-Pods sicher ($10 \times 35 = 350$). Der Worker-HPA bekommt deshalb `maxReplicas: 10`. Erst eine bewusste DB-Skalierung oder PgBouncer-Entscheidung darf diese Grenze erhöhen.

8.6 Kostenkontrolle

- Dev und Loadtest getrennte Environments/States; Loadtest per zeitlich begrenztem Runbook hoch- und herunterfahren.
- Cloud SQL HA und regionale GKE-Cluster kosten auch ohne Last. Für Dev kleinere/zonalere Varianten zulassen, aber prod-like nicht schönrechnen.
- Budgets benachrichtigen, stoppen Ressourcen aber nicht automatisch. TTL-/Cleanup-Automation für temporäre Runner.
- Labels `env`, `service`, `owner`, `managed-by=terraform`, `cost-center` auf Ressourcen.

9. Priorisierte Restarbeit und Definition of Done

9.1 Kritischer Pfad

Reihenfolge	Arbeitspaket	Blockiert
1	Production-Config, Reset/CORS, Probes, Redis TIME	Sicheren Multi-Pod-Betrieb
2	Dockerfiles + Migration Image	Jeden GKE-Deploy
3	Terraform Bootstrap + Netzwerk	Alle privaten Managed Services
4	Cloud SQL + Redis + Pub/Sub + IAM	App-Connectivity
5	GKE Baseline + Gateway + Observability	Öffentlichen End-to-End Flow
6	Lease Leader Election + HPA	Worker-HA und echte Skalierung
7	Cloudfähiger Test-Harness	Gültigen Cloud-Kapazitätsnachweis
8	Cloud-Loadtest + Report	Abschluss Phase 5

9.2 Was vor Phase 5 noch aus Phase 4 offen ist

- Der lokale Kapazitätsnachweis bleibt ungültig, solange Generator und SUT nicht getrennt sind.
- Baseline D mit erhöhtem `DATABASE\_POOL\_MAX` fehlt.
- Transportfehler unter hoher lokaler Last sind noch nicht abschließend untersucht.
- Die Report-Automation ist lokal-only: Container-Checks, `docker exec`, Emulator-Seed und einzelner k6-Prozess.

Diese Punkte widersprechen einem Cloud-Start nicht. Im Gegenteil: Die Cloud-Plattform kann die saubere Trennung von Generator und SUT herstellen. Aber der Test-Harness muss vor dem finalen Phase-5-Run cloudfähig werden.

9.3 Definition of Done für Phase 5

Gate	Erfüllt, wenn ...
IaC	Ein frisches Environment entsteht reproduzierbar aus dokumentiertem Bootstrap + Terraform Apply; State liegt remote und Provider-Versionen sind gelockt.
Security	Keine statischen SA-Keys, private SQL/Redis, least privilege KSAs, Reset nicht öffentlich, CORS eng, Secrets nicht im Repo/State.
Images	Drei Non-Root Images starten per Digest, bestehen Health/SIGTERM-Smoke und sind gescannt.
Kubernetes	Probes, Requests, HPA, PDB, Topology Spread, SecurityContext, NetworkPolicy und Gateway sind wirksam getestet.
Stateful Services	Cloud SQL HA/Backup/PITR und Memorystore HA/Memory Policy sind dokumentiert und praktisch erreichbar.
Correctness	Multi-API Unlock nutzt Redis TIME; mehrere Worker haben genau einen Reconcile-Leader; Failover wurde beobachtet.
Observability	Logs, App-Metriken, GKE-, SQL-, Redis- und Pub/Sub-Metriken sind zentral sichtbar; Alerts haben Runbooks.
Delivery	GitHub OIDC baut, scannt und deployt unveränderliche Images; Migration und Rollback sind kontrolliert.
Evidence	Cloud-Lasttest hat gültigen Generator, grüne Invarianten, archivierte Rohdaten und einen README-tauglichen Report.

9.4 Meine Architekturentscheidung in einem Satz

Ein regionaler, privater GKE-Standard-Cluster betreibt stateless Web/API und horizontal skalierte Worker; Cloud SQL, Memorystore und Pub/Sub halten den Zustand; Terraform besitzt die Plattform; Gateway API und Managed Prometheus liefern Edge und Observability; Workload Identity verbindet alles ohne Schlüsseldateien.

Nicht jetzt

Noch kein Multi-Region-GKE, kein Service Mesh, kein Spanner, kein selbst betriebenes PostgreSQL/Redis, kein GitOps-Operator und kein vollständiges Platform-Team-Modell. Diese Dinge erhöhen die Lernfläche, aber lösen die aktuellen Phase-5-Risiken nicht.

10. Lernlabor: Aufgaben mit abnehmender Hilfestellung

Level 1 - Erklären

- Zeichne die drei Ebenen Plattform, Workloads und Managed State aus dem Gedächtnis.
- Erkläre, warum die API keinen Cloud-SQL-IAM-Grant bekommt.
- Erkläre den Unterschied zwischen Terraform State und Kubernetes desired state.
- Begründe, warum Pub/Sub At-Least-Once plus DB-Idempotenz hier sinnvoll ist.

Level 2 - Anwenden

- Leite für `api`, `worker` und `migrator` jeweils die minimalen IAM-Rollen ab.
- Berechne drei Worker-HPA-Obergrenzen für DB-Budgets von 200, 400 und 800 Connections.
- Formuliere Readiness-Kriterien für API und Worker und simuliere je einen Abhängigkeitsausfall.
- Entwirf einen CIDR-Plan mit getrennten Node-, Pod-, Service- und PSA-Ranges.

Level 3 - Entscheiden

- Vergleiche Autopilot und Standard für Dev, Loadtest und Production; dokumentiere deine Abweichung von der Empfehlung.
- Entscheide, ob Redis AUTH/TLS in Phase 5 zwingend ist, und notiere die nötigen Code-/Secret-Änderungen.
- Definiere ein Scale-up- und Scale-down-Verhalten für den Worker anhand Backlog und oldest unacked age.
- Schreibe eine Rollback-Entscheidungstabelle für App-Fehler, Migration-Fehler und Infrastruktur-Fehler.

Level 4 - Beweisen

- Töte den aktiven Reconcile-Leader und beweise, dass genau ein anderer Pod übernimmt.
- Lass Redis ausfallen und beweise: API wird unready, aber nicht durch Liveness dauerhaft neugestartet.
- Führe einen Deploy während Last aus und beweise, dass PDB, RollingUpdate und Readiness 5xx verhindern.
- Stelle Cloud SQL aus PITR in eine neue Instanz wieder her und dokumentiere RTO/RPO.
- Führe den Cloud-Lasttest zweimal mit identischer Konfiguration aus und erkläre die Varianz.

Selbsttest

Frage	Gute Antwort enthält
Wer erzeugt ein Pub/Sub Topic?	Terraform; nie API/Worker beim Boot.
Wer startet zusätzliche API-Pods?	HPA ändert Deployment replicas; Scheduler platziert Pods; Cluster Autoscaler stellt Nodes bereit.
Wer schützt vor doppeltem Ticketverkauf?	DB-Transaktion/Unique orderId; Redis processed ist nur Fast Path.
Warum ist Worker-HPA riskanter als API-HPA?	Jeder Worker bringt DB-Pool und Reconcile-Verhalten mit; externe Connection-Grenze.
Warum reicht ein grünes `/health` nicht?	Es trennt Prozessleben und Traffic-Bereitschaft nicht.
Warum Image Digest statt Tag?	Digest ist unveränderlich und macht Rollout/Rollback reproduzierbar.
Was beweist 50k RPS nicht?	Nicht automatisch Korrektheit, Drain-SLO, Generator-Gesundheit oder Kostenverträglichkeit.

11. Glossar

Begriff	Bedeutung im Projekt
Artifact Registry	Private Registry für die drei OCI-Images.
Cloud NAT	Kontrollierter Internet-Egress für private GKE-Nodes.
Cloud SQL Auth Proxy	Sidecar, das IAM-authentisierte, TLS-geschützte DB-Verbindungen anbietet.
Control Plane	Controller, APIs und Terraform, die Sollzustand verwalten.
Data Plane	Pods und Dienste, die echte Requests/Nachrichten verarbeiten.
Gateway API	Kubernetes-API für Load Balancing, Routing und TLS.
HPA	Horizontal Pod Autoscaler; ändert Replica-Zahl anhand Metriken.
KSA	Kubernetes Service Account, Identität eines Pods im Cluster.
Lease	Kubernetes-Objekt für zeitlich begrenzte Leader-Election-Führung.
PDB	Pod Disruption Budget; begrenzt freiwillige gleichzeitige Ausfälle.
PITR	Point-in-Time Recovery einer Datenbank auf einen früheren Zeitpunkt.
Private Service Access	Private VPC-Verbindung zu gemanagten Service-Produzenten wie SQL/Redis.
RPO	Maximal akzeptierter Datenverlust in Zeit.
RTO	Maximal akzeptierte Wiederherstellungsdauer.
State	Terraform-Zuordnung zwischen Konfiguration und realen Ressourcen.
Topology Spread	Verteilt Pods über Zonen/Nodes, um gemeinsame Ausfälle zu reduzieren.
Workload Identity	Kurzlebige, föderierte GCP-Identität für Pods oder CI ohne JSON-Key.

12. Quellen und Repo-Belege

Analysezeitpunkt: 29. Juli 2026. Externe technische Aussagen wurden gegen offizielle Google-Cloud-Dokumentation abgeglichen. Produktpreise und konkrete GKE-Versionen sind absichtlich nicht festgeschrieben, weil sie sich ändern.

12.1 Repo-Belege

[R1] ``README.md`` - Systemziel, Stack und lokale Architektur.

[R2] ``docs/TODO.md`` - Roadmap, Phase 5 und offener Cloud-/Lasttest-Backlog.

[R3] ``docs/ARCHITECTURE.md`` - Kauf-Flow, Redis-Key-Lifecycle, Reconcile-Betriebsmodell.

[R4] ``docs/DECISIONS.md`` - ADR-010 Terraform, ADR-022 Reconcile, ADR-024 Sale Unlock, ADR-028 Payment Split.

[R5] ``packages/env/src/index.ts``, `API/Worker-Plugins` und Kubernetes-relevante Runtime-Verträge.

[R6] ``docs/reports/baseline-c-2026-07-26/LOAD-TEST-REPORT-2026-07-26.md`` - Durchsatz, Korrektheit und Engpässe.

12.2 Offizielle Dokumentation

[GCP-1] [GKE Autopilot overview](#)

[GCP-2] [Cloud SQL Auth Proxy](#)

[GCP-3] [Connect Memorystore Redis from GKE](#)

[GCP-4] [Private Service Access for Memorystore](#)

[GCP-5] [Cloud SQL private IP](#)

[GCP-6] [Memorystore for Redis overview](#)

[GCP-7] [Workload Identity Federation for GKE](#)

[GCP-8] [Terraform security best practices on Google Cloud](#)

[GCP-9] [Workload Identity Federation](#)

[GCP-10] [Cloud SQL availability](#)

[GCP-11] [Cloud SQL high availability and recovery](#)

[GCP-12] [Pub/Sub exactly-once delivery semantics](#)

[GCP-13] [Artifact Registry overview](#)

[GCP-14] [Artifact Registry image names and digests](#)

[GCP-15] [Secret Manager add-on for GKE](#)

[GCP-16] [GKE Gateway API](#)

[GCP-17] [Managed Service for Prometheus](#)

[GCP-18] [Managed Prometheus collection](#)

[GCP-19] [GKE Horizontal Pod Autoscaling](#)

[GCP-20] [GKE traffic-based autoscaling](#)

Dieses Dokument ist ein Architektur- und Lernartefakt. Vor dem ersten ``terraform apply`` müssen Projekt-ID, Region, Billing, Domain, Sicherheitsanforderungen und Kostenrahmen konkret gewählt werden.